

DevOps Incident Dashboard – End-to-End Docker to Kubernetes Deployment

Kaushal Kumar · B.Tech CSE · Environment: RHEL 10 + Minikube

Concise, technical viva presentation describing an end-to-end workflow from local Docker image to Kubernetes cluster deployment, service exposure, and basic incident observability.



Project Overview

What it is

A small Node.js/Express web application packaged in Docker and deployed to a Minikube Kubernetes cluster with a NodePort service and basic health checks.

Why it was built

To demonstrate the full DevOps lifecycle—build, package, publish, deploy, expose, and monitor—within a reproducible local environment.

Objective

Provide a teachable incident dashboard that supports health endpoints and validates container lifecycle and Kubernetes probes.

Problem solved

Simplifies learning by combining containerization, image registry, cluster orchestration, service exposure, and basic runtime probes in one workflow.

End-to-end learning goal

Equip students with practical skills: Dockerfile authoring, image lifecycle, kubectl usage, manifest authoring, probe configuration, and network access via NodePort.

Technologies & Roles



VirtualBox

Provides an isolated VM host for running RHEL10 when native virtualization is required; ensures consistent environment for Minikube VM driver.



RHEL 10

Base OS for the development VM. Reproducible environment for package management, Docker engine installation, and Minikube compatibility.



Docker

Builds and runs container images locally. Used to create the application image that will be deployed to Kubernetes.



Docker Hub

Registry for publishing images (e.g., kaushal13x/devops-dashboard:v1). Enables pulling images into the cluster when using a non-local registry setup.



Minikube

Local single-node Kubernetes cluster for development and testing; runs the deployed Pods and Services used in demonstrations.



kubectl

CLI to interact with the cluster: apply manifests, inspect Pods, port-forward, describe resources, and view logs for incident triage.



Node.js + Express

Lightweight web app framework providing endpoints for health, version, and uptime—used to exercise container lifecycle and probes.

Application Architecture



Request flow: Client browser → NodePort service forwards to Pod selector → kube-proxy routes to one of the replica Pods → container receives request → Node.js handles route.

Replicas and HA: Deployment configured with replicas=2 ensures pod redundancy; Kubernetes restarts failed containers and re-schedules on node failure within Minikube limits.

Application Features & Operational Endpoints

/health

Lightweight readiness indicator returning simple JSON {status: 'ok'} used for quick health checks and uptime alerting.

/version

Returns application semantic version and build metadata. Useful for verifying deployed image tag (e.g., v1) during incident investigation.

/uptime

Reports process uptime and start timestamp; helps confirm container restarts or crash-loop behaviors when correlated with Kubernetes events.

Environment variables

Configure runtime behavior (PORT, NODE_ENV, APP_VERSION).
Kubernetes passes values via container.env to avoid baking secrets or config into the image.

Liveness probe

Probe ensures container health; failing livenessProbe triggers container restart—used to simulate recovery from transient faults during demos.

Docker Implementation



- Dockerfile base image: node:18-alpine – minimal, secure, fast startup.
- WORKDIR: create application working directory inside container to standardize paths.
- COPY: application source (package.json, package-lock.json, src) into the image.
- RUN: npm install – installs production dependencies at build time.
- EXPOSE 3000: declare listening port for clarity and documentation (runtime uses env PORT).
- CMD: start Node.js app (e.g., node index.js or npm start).
- Image tagging convention: kaushal13x/devops-dashboard:v1 – follows registry/repo:tag format for Docker Hub.

Key Docker Commands

1 `docker build -t kaushal13x/devops-dashboard:v1 .`

Builds the image locally from the Dockerfile in the current directory and tags it for pushing to Docker Hub.

2 `docker images`

Lists local images and confirms the presence of the newly built tag and image ID for verification.

3 `docker login`

Authenticate to Docker Hub to allow pushing images; required before docker push when using remote registry.

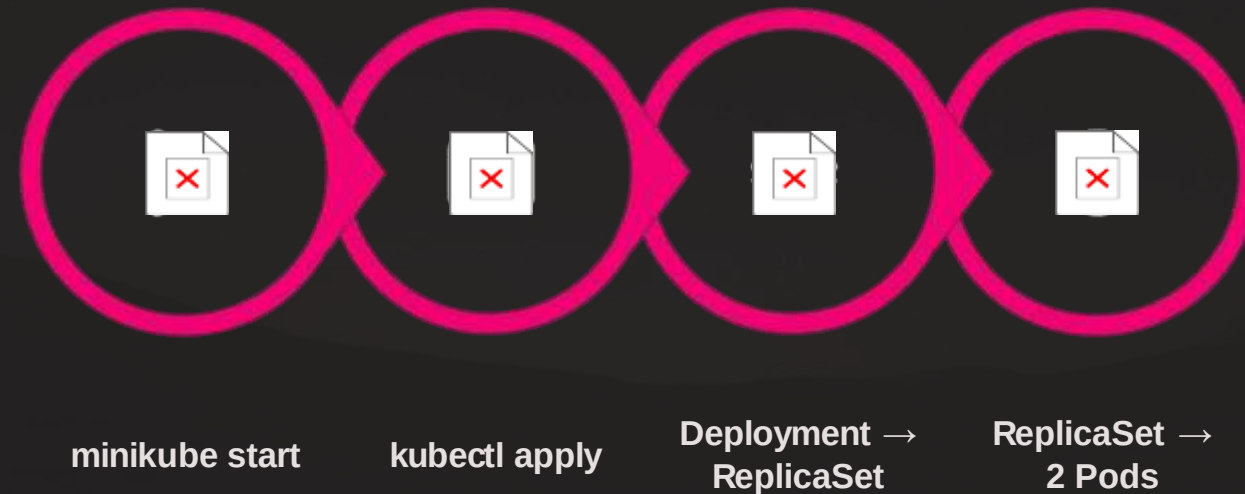
4 `docker push kaushal13x/devops-dashboard:v1`

Uploads the image to Docker Hub so Kubernetes can pull it (if the cluster cannot access the local Docker daemon).

5 `docker run -p 3000:3000 <image-name>`

Run container locally for functional testing before building manifests. Maps host port 3000 to container port 3000 for browser access.

Kubernetes Deployment (Minikube)



Manifest highlights (deployment.yaml):

- `apiVersion: apps/v1, kind: Deployment` – declarative workload object.
- `spec.replicas: 2` – maintains two pod replicas for redundancy and basic HA.
- `spec.template.spec.containers[].image: kaushal13x/devops-dashboard:v1` – image reference.
- `containerPort: 3000` – documents the container's listening port for readiness and probes.
- `env: pass PORT, NODE_ENV, APP_VERSION` – decouples configuration from image.
- `livenessProbe: httpGet {/health}`
`initialDelaySeconds/readinessProbe as appropriate` – detects unhealthy containers and restarts them automatically.

Command: `minikube start → kubectl apply -f deployment.yaml`. Use `kubectl get pods`, `kubectl describe pod`, and `kubectl logs` for runtime inspection and incident triage.

Service Exposure & Networking

service.yaml

Defines a Service of kind: Service with spec.type: NodePort to expose the application outside the cluster on a specific node port.

type: NodePort

Allocates a port on each node (nodePort) to forward traffic to targetPort (container port). Simple approach for local clusters like Minikube.

Port mapping

Service port 80 → targetPort 3000 (container). nodePort configured as 30007 to provide a fixed endpoint:
`http://<minikube-ip>:30007`

Access using minikube ip

Retrieve IP via minikube ip; combine with nodePort to access the app from host machine or external tester.

How traffic routes

kube-proxy programs iptables rules to route nodePort traffic to Service endpoints; Service uses Pod selectors to load-balance requests across replicas.